1 | Welcome

1.1 Introduction

Thank you for having put your faith on this book. If you want to learn how to use a powerful tool that allows developers to quickly create native applications with top performances, you've chosen the right book. Nowadays companies tend to consider cross-platform solutions in their development stack mainly for three reasons:

1. **Faster development**: working on a single codebase;
2. **Lower costs**: maintaining a single project instead of many (N projects for N platforms);
3. **Consistency**: the same UI and functionalities on any platform.

All those advantages are valid regardless the framework being used. However, for a complete overview, there's the need to also consider the other side of the coin because a cross-platform approach also has some drawbacks:

1. **Lower performances**: a native app can be slightly faster thanks to the direct contact with the device. A cross-platform framework might produce a slower application due to a necessary *bridge* required to communicate with the underlying OS;
2. **Slower releases**: when Google or Apple announce a major update for their OS, the maintainers of the cross-platform solution could have the need to release an update to enable the latest features. The developers must wait for an update of the framework, which might slow down the work.

Every framework adopts different strategies to maximize the benefits and minimize or get rid of the drawbacks. The perfect product doesn't exist, and very likely we will never have one, but there are some high quality frameworks you've probably already heard:

- **Flutter**. Created by Google, it uses Dart;

- **React Native**. Created by Facebook, it is based on javascript;
- **Xamarin**. Created by Microsoft, it uses the C#;
- **Firemonkey**. Created by Embarcadero, it uses Delphi.

During the reading of the book you will see how Google tries to make the cross-platform development production-ready using the Dart programming language and the Flutter UI framework. You will learn that Flutter renders everything by itself [1] in a very good way and it doesn't use any intermediate *bridge* to communicate with the OS. It compiles directly to ARM (for mobile) or optimized JavaScript (for web).

1.1.1 Who is this book for

To get the most out of this book, you should already know the basics of object-oriented programming and preferably at least an "*OOP language*" such as Java or C#. Our goal is trying to make the contents of this book understandable for the widest possible range of developers. Nevertheless, you should already have a minimum of experience in order to better understand the concepts.

If you already know what is a class, what is inheritance and what is nullability, part 1 of this book is going to be a walk in the park. Foreknowledge aside, we will talk about both Dart and Flutter "from scratch" so that the reader can understand any concept regardless the expertise level.

1.1.2 Author

Alberto Miola is an Italian software developer that started working with Delphi (Object Pascal) for desktop development and Java for back-end and Android apps. He currently works in Italy where he daily uses Flutter for mobile and Java for desktop and back-end. Alberto graduated

[1] For example, it doesn't use the system's OEM widgets

in computer science at University of Padua with a thesis about cross-platform frameworks and OOP programming languages.

1.1.3 Acknowledgments

This book owes a lot to some people the author has to mention here because he thinks it's the minimum he can do to express his gratitude. They have technically supported the realization of this book with their fundamental comments and critiques that improved the quality of the contents.

- **Rémi Rousselet**. He is the author of the famous "provider" [2] package and a visible member in the Flutter/Dart community. He actively answers on stackoverflow.com helping tons of people and constantly works in the creation of open source projects.
- **Felix Angelov**. Felix is a Senior Software Engineer at Very Good Ventures. He previously worked at BMW for 3 years and is the main maintainer of the bloc state management library. He has been building enterprise software with Flutter for almost 2 years and loves the technology as well as the amazing community.
- **Matej Rešetár**. He is helping people get prepared for real app development on resocoder.com and also on the Reso Coder YouTube channel. Flutter is an amazing framework but it is easy to write spaghetti code in it. That's why he's spreading the message of proper Flutter app architecture.

Special thanks to my friends Matthew Palomba and Alfred Schilken which carefully read the book improving the style and the quality of the contents.

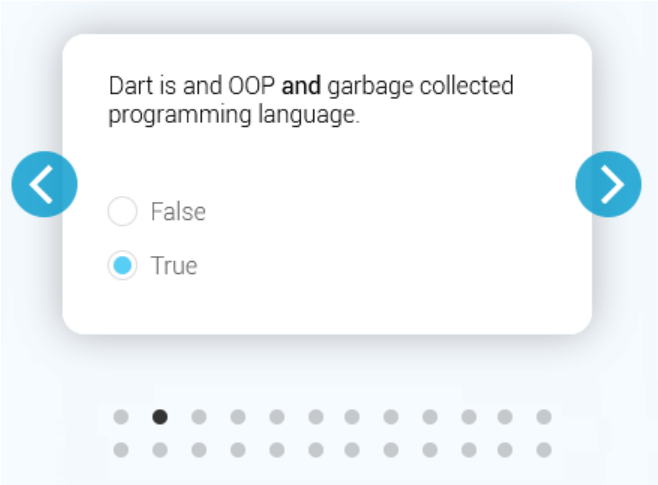
1.1.4 Online resources and the quiz

The official website of this book [3] contains the source code of the examples described in Part III. While reading the chapters you might encounter this box:

Resources > Chapter 16 > Files download

[2] https://pub.dev/packages/provider
[3] https://fluttercompletereference.com

It indicates that if you navigate to the *Resources* page of our website, you'll find the complete source code of the example being discussed at *Chapter 16 > Files download*. In addition, you can play the "Quiz game" which will test the Dart and Flutter skills you've acquired reading this book.



At the end, the result page will tell you the exact page of the book at which you can find an explanation of the answer.

1.2 Introduction to Dart

Dart is a client-optimized, garbage-collected, OOP language for creating fast apps that run on any platform. If you are familiar with an object oriented programming language such as Java or C# you might find many similarities with Dart. The first part of this book aims to show how the language can help you solving problems and the vastness of its API.

1.2.1 Supported platforms

Dart is a very flexible language thanks to the environment in which it lives. Once the source code has been written (and tested) it can be deployed in many different ways:

- **Stand-alone**. In the same way as a Java program can't be run without the Java Virtual Machine (JVM), a stand-alone Dart program can't be executed without the Dart Virtual Machine (DVM). There's the need to download and install the DVM which to execute Dart in a command-line environment. The SDK, other than the compiler and the libraries, also offers a series of other tools:
  - the `pub` package manager, which will be explored in detail in chapter 23;
  - `dart2js`, which compiles Dart code to deployable JavaScript;
  - `dartdoc`, the Dart documentation generator;
  - `dartfmt`, a code formatter that follows the official style guidelines.

  In other words, with the stand-alone way you're creating a Dart program that can only run if the DVM is installed. To develop Flutter apps for any platform (mobile, web and desktop), instead of installing the "pure" Dart SDK, you need to install Flutter [4] (which is basically the Dart SDK combined with Flutter tools).

- **AOT compiled**. The **A**head **O**f **T**ime compilation is the act of translating a high-level programming language, like Dart, into native machine code. Basically, starting from the Dart source code you can obtain a single binary file that can execute natively on a certain operating system. AOT is really what makes Flutter fast and portable.

  With AOT there is **NO** need to have the DVM installed because at the end you get a single binary file (an *.apk* or *.aab* for Android, an *.ipa* for iOS, an *.exe* for Windows...) that can be executed.

  - Thanks to the Flutter SDK you can AOT compile your Dart code into a native binary

[4] https://flutter.dev/docs/get-started/install

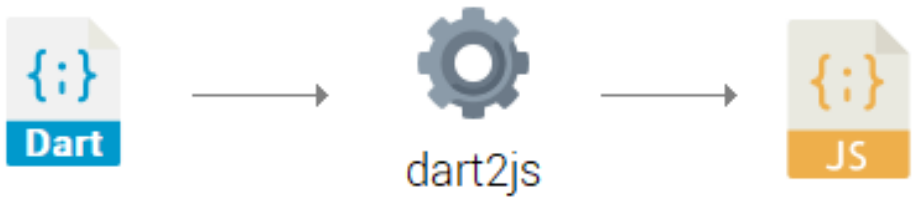
for mobile, web and desktop.

- As of Flutter 1.21, the Dart SDK is included in the Flutter SDK so you don't have to install them separately. They're all bundled in a single install package.

- Starting from version 2.6, the `dart2native` command (supported on Windows, macOS and Linux) makes AOT compiles a Dart program into x64 native machine code. The output is a standalone executable file.

AOT compilation is very powerful because it natively brings Dart to mobile desktop. You'll end up having a single native binary which doesn't require a DVM to be installed on the client in order to run the application.

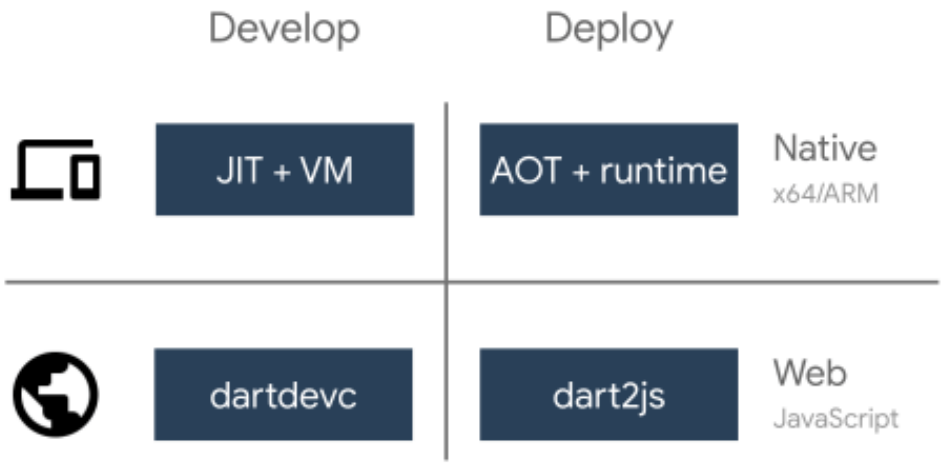
- **Web**. Thanks to the `dart2js` tool, your Dart project can be "transpiled" into fast and compact JavaScript code. By consequence Flutter can be run, for example, on Firefox or Chrome and the UI will be identical to the other platforms.



AngularDart [5] is a performant web app framework used by Google to build some famous websites, such as "AdSense" and "AdWords". Of course it's powered by Dart!

So far we've covered what you can do with Dart when it comes to deployment and production-ready software. When you have to debug and develop, both for desktop/mobile and web, there are useful some tools coming to the rescue.

[5] https://angulardart.dev/



This picture sums up very well how the Dart code can be used in development and deployment. We've just covered the "Deploy" side in the above part, so let's analyze the "Develop" column:

- **Desktop/mobile**. The **J**ust **I**n **T**ime (JIT) technique can be seen as a "real time translation" because the compilation happens while the program is executing. It's a sort of "dynamic compilation" which happens while the program is being used.

  JIT compilation, combined with the DVM (JIT + VM in the picture), allows the dispatch of the code dynamically without considering the user's machine architecture. In this way it's possible to smoothly run and debug the code everywhere without having to mess up with the underlying architecture.

- **Web**. The Dart development compiler, abbreviated with **dartdevc**, allows you to run and debug Dart web apps on Google Chrome. Note that dartdevc is for development only: for deployment, you should use **dart2js**. Using special tools like *webdev* [6] there's the possibility to edit Dart files, refreshing Chrome and visualizing changes almost immediately.

As you've just seen, Dart can run literally everywhere: desktop, mobile and web. This book will give you a wide overview of the language (Dart version 2.10, with null safety support) and all the required skills to create easily maintainable projects.

[6] https://dart.dev/tools/webdev#serve

1.2.2 Package system

Dart's core API offers different packages, such as `dart:io` or `dart:collection`, that expose classes and methods for many purposes. In addition, there is an official online repository called *pub* containing packages created by the Dart team, the Flutter team or community users like you.

If you head to https://pub.dev you will find an endless number of packages for any purpose: I/O handling, XML serialization/de-serialization, localization, SQL/NoSQL database utilities and much more.

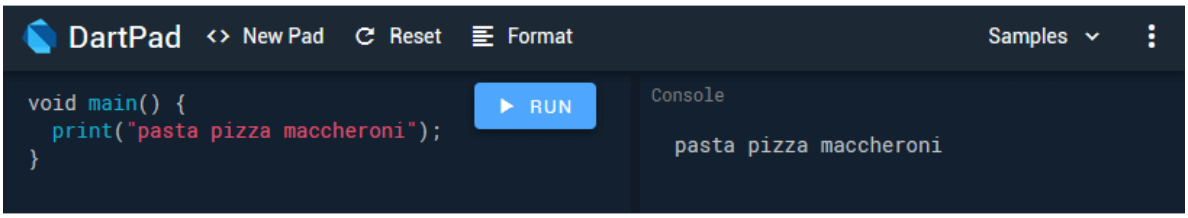
1. Go to https://pub.dev, the official repository;
2. Let's say you're looking for an equation solving library. Type "equations" in the search bar and filter the results by platform. Some packages are available only for Dart, others only for Flutter and a good part works for both;
3. The page of the package contains an installation guide, an overview and a guide so that you won't get lost.

You should check the amount of *likes* received by the community and the overall *reputation* of the package because those values indicate how mature and healthy the product is. You will learn how to properly write a library and how to upload it to the pub.dev repository in order to give your contribution to the growth of the community.

1.2.3 Hello World

The simplest way you have to run your Dart code is by opening DartPad [7], an open-source compiler that works in any modern browser. Clicking on "New Pad" you can decide whether creating a new Dart or Flutter project (with latest stable version of the SDK).

[7] https://dartpad.dartlang.org/



It's the perfect tool for the beginners that want to play with Dart and try the code. If you're new to the language, start using DartPad (which is absolutely **not** and IDE). It always has the latest version of the SDK installed and it's straightforward to use.

```
void main() {
    // Best food worldwide!
    print("pasta pizza maccheroni");
}
```

Like with Java and C++, any Dart program has to define a function called `main()` which is the entry point of the application. Very intuitively the `print()` method outputs to the console, on the right of the DartPad, a string. Starting from chapter 2, you'll begin to learn the syntax and the good practices that a programmer should know about Dart.

Android Studio

Visual Studio Code

When you develop for real world applications, you're going to download the whole SDK and use an IDE like IntelliJ IDEA, Android Studio or VS Code. DartPad doesn't give you the possibility to setup tests, import external packages, add dependencies and test your code.

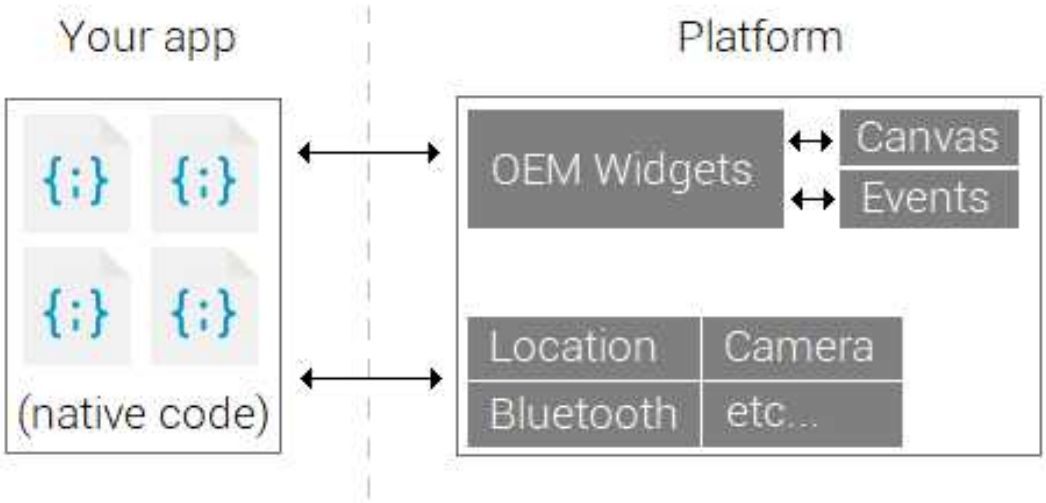
1.3 Intorduction to Flutter

Flutter is an UI toolkit for building natively compiled applications for mobile, desktop and web with a single codebase. At the time of writing this book, only Flutter for mobile is stable and ready for production. Web support is currently in beta while desktop (macOS, Linux and Windows) is in early alpha: they will be covered in a future release of this reference once they will be officially released as stable builds.

Being familiar with *Jetpack compose* or *React Native* is surely an advantage because the concepts of reactive views and "components tree" are the fundamentals of the Flutter framework.

1.3.1 How does it work

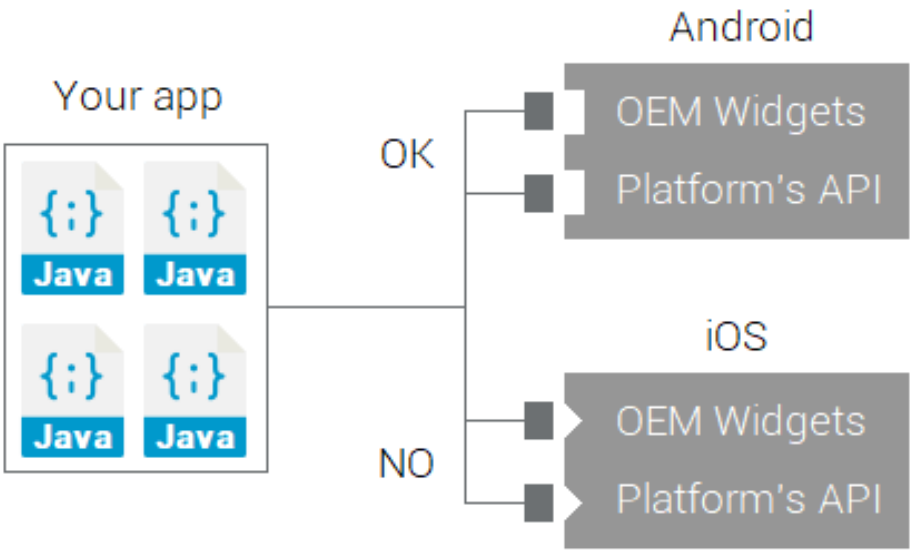
This picture shows how a native app interacts with the OS, whether it's been written in Kotlin (or Java) for Android or Swift (or Objective-C) for iOS. We're going to use these 2 platforms as examples in this section.



1. The platform, which can be Android or iOS, exposes a series of OEM widgets used by the app to build the UI. Those widgets are **fundamental** because they give our app the capabilities to paint the UI, use the canvas and respond to events such as finger taps.
2. If you wanted to take a picture from your app or use the bluetooth to send a file, there would be the need to communicate with the native API exposed by the platform. For example, using OS-specific APIs, you could ask for the camera service, wait for a response and then start using it.

The cross-platform approach is different and it **has** to be like so. If you want your app to run on both Android and iOS with the same codebase, you can't directly use OEM widgets and their API because they come from different architectures. They are **NOT** compatible. On the hardware side however, both are based on the ARM architecture (precisely, v7 and v8) and the most recent versions have 64-bit support. Flutter AOT compiles the Dart code into native ARM libraries.

> **ARM** is a family of RISC microprocessors (32 and 64 bit) widely used in embedded systems. It dominates the mobile world thanks to its qualities: low costs, good heat dissipation and a longer battery life thanks to a low power consumption.

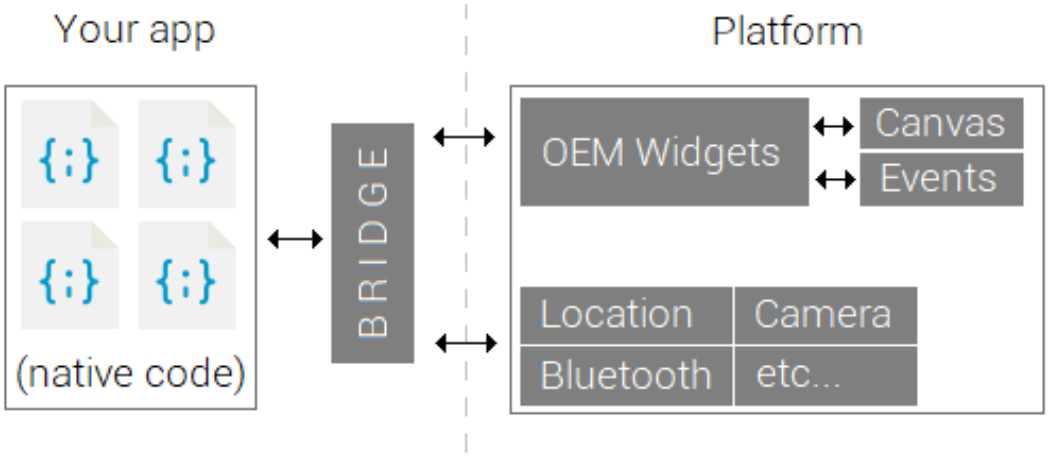


The picture has rectangles on Android and triangles on iOS to indicate that OEM widgets and

APIs have differences in how they are structured, in how they interact with the app and in how you have to use them. For this reason, cross-platform apps cannot directly "talk" to the underlying environment: they must speak a language that everyone can understand.

> Try to only think about the runtime environment for a moment. If you wrote a Java Android app, it would be compiled to work with the ART ecosystem (**A**ndroid **R**un**T**ime): how could the same binary file work with iOS architecture which is completely different and has no ART?
>
> In the above image, squares represents calls made by Java to interact with the ART which is available only in Android and not on iOS. This compatibility problem is solved by cross-platform frameworks.

ReactJS is a Reactive web framework which tries to solve the above problem by adding a *bridge* in the middle that takes care of the communication with the platform. With this approach, the bridge becomes the real starring of the scene it acts like a translator:

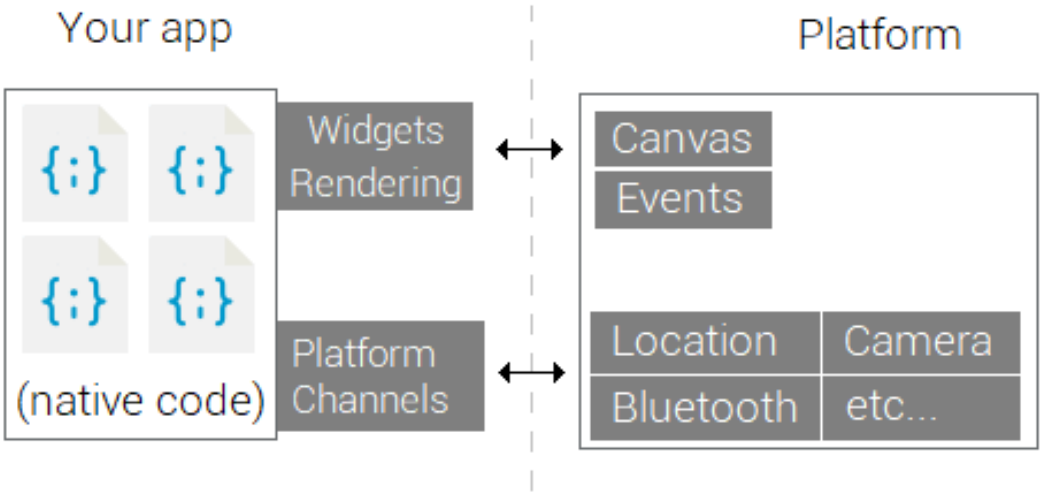


1. The bridge **always** exposes the same interface to the app so that it doesn't care anymore about the OS it's running on;
2. The bridge has an implementation of OEMs and APIs for each platform to allow the app to correctly work in many environments. In this way, you have a native app in the sense that it uses the native tools given by the OS, but there's still an "adapter" in the middle.

As you can see from the picture, the *bridge* is an abstraction layer between the app the OS in which it's hosted. Of course, there has to be a bridge for each supported platform but that's not something you have to deal with because the developers of the framework will take care of creating all of them.

> ℹ If you used a cross-platform framework, you'd just need to care about creating the app with the code and the API exposed by the framework. The implementation of the bridge is already in the internals of the SDK and it's automatically "attached" to the app in the build phase. You don't have to create the bridge.

The *bridge* approach is quite popular, but it could be a potential bottleneck that slows down the execution and thus the performances might drop. If you think about animations, swipes or transitions, widgets are accessed very often and many of them running at the same time could slow down the app. Flutter adopts a completely different strategy:



It uses its own very efficient rendering engine, called *Skia*, to paint the UI so that OEM widgets are not needed anymore. In this way, the app doesn't rely on the instruments the OS exposes to draw the interface and you can freely control each single pixel of the screen.

- Flutter produces **native** ARM code for the machine;

- when launched, the app loads the Flutter library. Any rendering, input or event handling, and so on, is delegated to the compiled Flutter and app code. This is much faster than having a bridge.
- A minimal Flutter app is about 4.4 MB on Android and 10.9 MB on iOS (depending on the architecture, whether it be ARM 32 or 64 bit) [8]

The true power of Flutter lies on the fact that apps are built with their own rendering stuff and they are not constrained to paint the UI following the rules "imposed" by OEM widgets. You're free to control the screen and manipulate every single pixel.

1.3.2 Why Flutter uses Dart

There are many reasons behind the decision made by Google to choose Dart as language for the Flutter framework. At the time of writing this book, the latest **stable** version of Dart is 2.9.2 (Dart 2.10 is on beta, but downloadable anyway). Here's a summary [9] of what brought them to make this choice.

1. **OOP style**. The vast majority of developers have object-oriented programming skills and thus Dart would be easy to learn as it adopts most of the common OOP patterns. The developer doesn't have to deal with a completely new way of coding; he can reuse what he already knows and integrate it with the specific details of Dart.
2. **Performances**. In order to guarantee high performances and avoid frame dropping during the execution of the app, there's the need of a high performance and predictable language. Dart can guarantee to be very efficient and it provides a powerful memory allocator that handles small, short-lived allocations. This is perfect for Flutter's functional-style flow.
3. **Productivity**. Flutter allows developers to write Android, iOS, web and desktop apps with a single codebase keeping the same performances, aspect and feeling in each platform. A highly productive language like Dart accelerates the coding process and makes the framework more attractive.
4. Both Flutter and Dart are developed by Google which can freely decide what to do with them listening to the community as well. If Dart was developed by another company, Google probably wouldn't have the same freedom of choice in implementing new features and and the language couldn't evolve at the desired pace.

Another important aspect is that Dart is **strongly** typed, meaning that the compiler is going

[8] https://flutter.dev/docs/resources/faq#how-big-is-the-flutter-engine
[9] https://flutter.dev/docs/resources/faq#why-did-flutter-choose-to-use-dart

to be very strict about types; you'll have both less runtime surprises and an easier debugging process. In addition, keep in mind that Dart is a complete swiss-knife because it has built-in support for:

- tree-shaking optimization;
- hot reload feature;
- a package manager with mandatory documentation and the possibility to play with the code using DartPad;
- *DevTools*, a collection of debugging and performance tools;
- code documentation generator tool;
- support for JIT and AOT compilation.

By owning two home-made products, Google can keep the entire projects under control and decide how to integrate them in the best way possible with quick development cycles. Dart evolves together with Flutter and as time goes by: they help each other maximizing productivity and performances.

1.3.3 Hello world

When creating Flutter apps for the production world, you should really consider using Android Studio or VSCode and install the respective plugins. They offer a debugger, hints, a friendly UI and powerful optimization tools we will explore in detail.

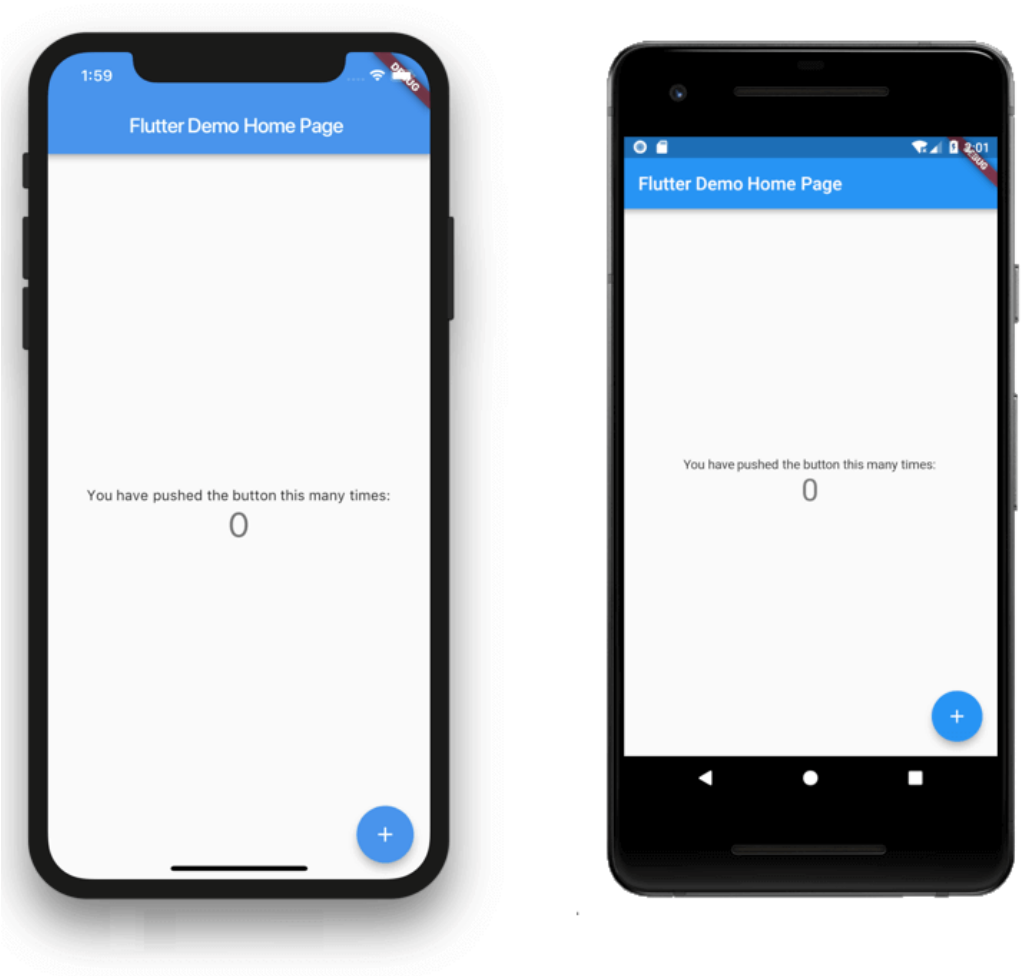
```
void main() {
    runApp(MyApp());
}

class MyApp extends StatelessWidget {
    const MyApp();

    Widget build(BuildContext context) {
        return MaterialApp(
            home: Scaffold(
                body: Center(
                    child: Text("Flutter app!"),
                ),
            ),
```

```
            );
        }
    }
```

This is a very simple example of a minimal Flutter application. You can notice immediately that there is a `void main() { ... }` function, required by Dart to define the entry point of the program. An UI is a composition of *widgets* that decorate the screen with many objects; you will learn how to properly use them to create efficient and beautiful designs.



This is an example of how a simple Flutter app looks identical in two different platforms. In this book we will focus on Android and iOS mobile apps but **everything** you're going to learn is also valid for web and desktop because it's always Flutter. Once you have the code ready, open the console...

```
$ flutter build appbundle

$ flutter build ios

$ flutter build web

$ flutter build macos

$ flutter build windows

$ flutter build linux
```

...and it's just a matter of running different *build* commands to get different native binaries of the same app. For more info on Flutter for web and desktop, see the appendix B at the bottom of the book.